

ROADMAP GRATIS · 30 DÍAS

De 0 a tu primer proyecto real con IA.

El método exacto que aplico con cada alumno para pasar de cero a un proyecto desplegado en 30 días.

Raúl Huete · Software Architect en activo

10 años escribiendo código en producción

LO QUE NADIE TE DICE

El problema NO es la información.

Llevo años hablando con personas que quieren aprender a programar. Y siempre detecto el mismo patrón:

- Han hecho 4-5 cursos.
- Ven YouTube todas las noches.
- Conocen los nombres de 10 frameworks.
- Y no han construido nada propio.

El problema NO es falta de información. Es exceso de información sin dirección.

**La diferencia entre alguien que avanza y alguien que abandona
NO es talento. Es tener un plan claro.**

LO QUE VAS A ENCONTRAR AQUÍ

Este PDF es ese plan.

30 días divididos en 4 fases. Cada fase con su objetivo concreto. Sin teoría innecesaria. Sin frases motivacionales. Sin promesas absurdas.

Lo que aplico con cada persona con la que trabajo 1:1, resumido en lo esencial.

Elegir tu stack.

Objetivo: salir de la indecisión.

El error #1 que veo: gente que lleva 6 meses sin empezar porque no sabe si elegir JavaScript, Python o Go.

Mi consejo es siempre el mismo: olvida los rankings de Twitter. La tecnología correcta para ti depende de TRES preguntas:

1. ¿Qué tipo de problemas quieres resolver?

Web, datos, móvil, automatización con IA. Cada uno tiene su stack natural.

2. ¿Dónde quieres trabajar dentro de un año?

Busca 20 ofertas en LinkedIn de las empresas que te gustaría. Anota las tecnologías que se repiten.

3. ¿Qué stack te lleva al primer proyecto funcionando más rápido?

Curva de aprendizaje > popularidad. Casi siempre.

Una vez elegido, NO mires atrás durante 6 meses.
El FOMO técnico es la razón #1 por la que la gente no avanza.

PREGUNTAS QUE QUEDAN ABIERTAS

→ ¿Y si las 3 preguntas te llevan a stacks distintos? ¿Cómo prioriza alguien con tu situación concreta?

→ ¿Cuáles son las tecnologías que estoy viendo crecer en el sector y nadie está aprendiendo todavía?

Construir un proyecto real.

Objetivo: cambiar de consumir a producir.

Aquí es donde mueren la mayoría de aprendices. Llevan 6 meses estudiando, saben mucho de teoría, pero NO han construido nada propio.

Tu cerebro NO aprende programación viendo a otros codificar. Aprende cuando tiene un problema que resolver y se siente incómodo intentándolo.

¿Qué tipo de proyecto?

- ✓ Que resuelva UN problema concreto
- ✓ Que se termine en máximo 10 días
- ✓ Que NO sea el próximo Netflix
- ✓ Que puedas explicar línea por línea

Ejemplos válidos:

- Dashboard para visualizar gastos personales
- API que automatice una tarea repetitiva tuya
- Chatbot con IA que responda preguntas sobre un PDF
- Web que recopile datos públicos y los muestre

Programar NO es consumir información.
Es resolver problemas con código.

PREGUNTAS QUE QUEDAN ABIERTAS

- ¿Qué proyecto concreto debería elegir YO según mi background y mi objetivo profesional?
- ¿Cómo descompongo ese proyecto en tareas diarias factibles con mi tiempo real?

Pulir y publicar.

Objetivo: pasar de amateur a profesional.

La diferencia entre código de aprendiz y código de profesional NO está en la lógica. Está en la PRESENTACIÓN.

Lo que tienes que cuidar:

1. README profesional en GitHub

Qué hace, cómo se ejecuta, capturas, tecnologías usadas, decisiones técnicas.

2. Naming de variables profesional

Nada de `data`, `x`, `function1`. Cada nombre debe explicar qué hace.

3. Manejo de errores básico

Si tu app crashea con un input inesperado, el reviewer cierra la pestaña.

4. Deploy público funcionando

Vercel, Railway o Render. Gratis. Sin URL pública, no existe.

**Si nadie puede VER tu código,
para el mercado no existe.**

PREGUNTAS QUE QUEDAN ABIERTAS

→ ¿Cómo revisa tu README, naming y manejo de errores alguien que ya está en producción?

→ ¿Qué decisiones técnicas son las que las empresas valoran en código de un junior?

Mostrar lo que sabes.

Objetivo: convertir tu proyecto en oportunidades reales.

Saber programar y conseguir trabajo son habilidades DISTINTAS. Y la mayoría de aprendices descubre esto demasiado tarde, después de meses de esfuerzo invisible.

Tu plan de visibilidad estos 5 días:

Día 26 — Post de LinkedIn técnico

Cuenta tu proyecto: problema, decisión técnica, qué aprendiste. Foto del resultado.

Día 27 — Demo grabada de 2 minutos

Vídeo en YouTube no listado. Explicas el proyecto en marcha. Lo añades a tu CV.

Día 28 — Conexiones estratégicas

10 conexiones nuevas en LinkedIn con perfiles del sector que te interesa.

Día 29 — Aplicar a 3 ofertas concretas

Personalizadas. Mensaje directo al hiring manager. Tu proyecto como evidencia.

Día 30 — Preparación de entrevista técnica

Practica explicar tu proyecto en 2 minutos. Anticipa las 4 preguntas clave.

Mostrarte profesionalmente no es opcional.
Es la mitad del trabajo.

PREGUNTAS QUE QUEDAN ABIERTAS

→ ¿Cuáles son las 4 preguntas que casi siempre te hacen en entrevistas para juniors?

→ ¿Cómo personalizo cada aplicación para que destaque sin parecer scripted?

BONUS · LAS 3 REGLAS DE ORO

Para usar IA sin destruir tu carrera.

La IA es la herramienta más poderosa que existe para aprender a programar. Y también la más rápida para volverte profesionalmente inútil si la usas mal.

Estas son las 3 reglas que aplico yo todos los días como Software Architect:

REGLA 01 — No copies sin entender

Si la IA te da código que no puedes explicar línea por línea, NO es tu código. Pídele explicaciones antes de pegar. Si no entiendes la explicación, pide ejemplos más simples. Si todavía no la entiendes, ESE es el concepto que toca aprender hoy.

REGLA 02 — Pregunta POR QUÉ, no solo CÓMO

Cuando preguntas "cómo hago X", la IA es Stack Overflow rápido. Cuando preguntas "por qué Y y no Z", se convierte en profesor senior personal. Esto cambia TODO.

REGLA 03 — Programa SIN IA 2h/semana

Sin Cursor. Sin ChatGPT. Sin Copilot. Es la única forma de medir qué sabes de verdad y qué estás delegando. Como ir al gimnasio sin máquinas: sin esfuerzo no hay músculo.

**El futuro NO es de los developers que escriben más código con IA.
Es de los que entienden mejor los sistemas.**

UNA ÚLTIMA COSA

Esto es el mapa.

Tú decides si caminas solo o acompañado.

Este PDF te da el plan. Pero un plan genérico nunca va a ser tan efectivo como uno hecho a medida para TU situación concreta.

Las personas que avanzan más rápido tienen tres cosas en común:

- Eligen UN stack y se quedan ahí 6 meses.
- Construyen proyectos propios desde la semana 2.
- Tienen a alguien revisando su trabajo en tiempo real.

Las dos primeras las puedes hacer solo. La tercera no.

Y es justamente la que más acelera el proceso.

Sesión Diagnóstico · 30 min

Te ayudo a aplicar este roadmap a TU situación concreta.

Sales con un plan personalizado de 30 días en PDF.

29€ · Se descuentan del primer mes si te quedas.

→ Reserva en zero2dev.es

¿QUÉ INCLUYE LA SESIÓN?

- Análisis de tu situación actual (background, tiempo real disponible, objetivos)
- Elección de stack adaptada a TU ciudad y objetivo profesional
- Definición de tu primer proyecto concreto
- Plan día por día para tus primeros 30 días
- Respuestas a las 6 preguntas abiertas de este PDF

Si no es el momento, conserva este PDF. Vuelve cuando te haga falta.

Y si te ha aportado algo, compártelo con quien creas que lo necesita.